CCDS Project Implementation

1. Introduction

This project “Subtitle Generator” is a core java application integrated with Azure cloud services that aims to recognize, clean and create subtitles for any type of audio/video file in a variety of languages.

Technology Stack:

- Python FastAPI server – whisper_server.py
- Java Maven – App.java, Database.java, BlobService.java
- VS Code IDE – with PostgreSQL extension for Azure
- Ubuntu 24.04 Linux
- Azure Cloud Services – Virtual Machine, Registry, Resource Group, Database for PostgreSQL flexible servers, Blob Storage, Container, Translators

Technical Specifications of my laptop:

- RAM – 16GB
- Storage – 512 GB
- CPU Cores – 2
- GPU – Intel Integrated Iris XE
- Processor – Intel i7
- OS: Windows 11 Pro

Azure Basic Setup:

- Create an student account in Azure to claim free credits worth Rs. 9095
- Log in to your Azure account using the registered credentials
- Create a resource group following the steps provided in the next section.
- Whenever, configuring any resource/service throughout this project, always set your “Subscription Account” → “My Student Trial”, “Region” → “AsiaPacific (Central India)” and “Availability Zone” → “No Preference”
- Throughout this document, while creating any/all resource, “Tags” tab/section has not been configured or has been intentionally skipped. It can be configured additionally, if you want to apply IAM access control policies for different users, user groups, resource groups, specific resources etc.
- You may even set “Cost Alerts” in order to get timely notifications when paid resources are overrunning the set budget.

Note: Keep a proper record/track of all the “Username, Password” , “ssh keys (.pem) files”, “Connection Strings”, “Endpoint” and “Public IPs” for quick reference whenever required.

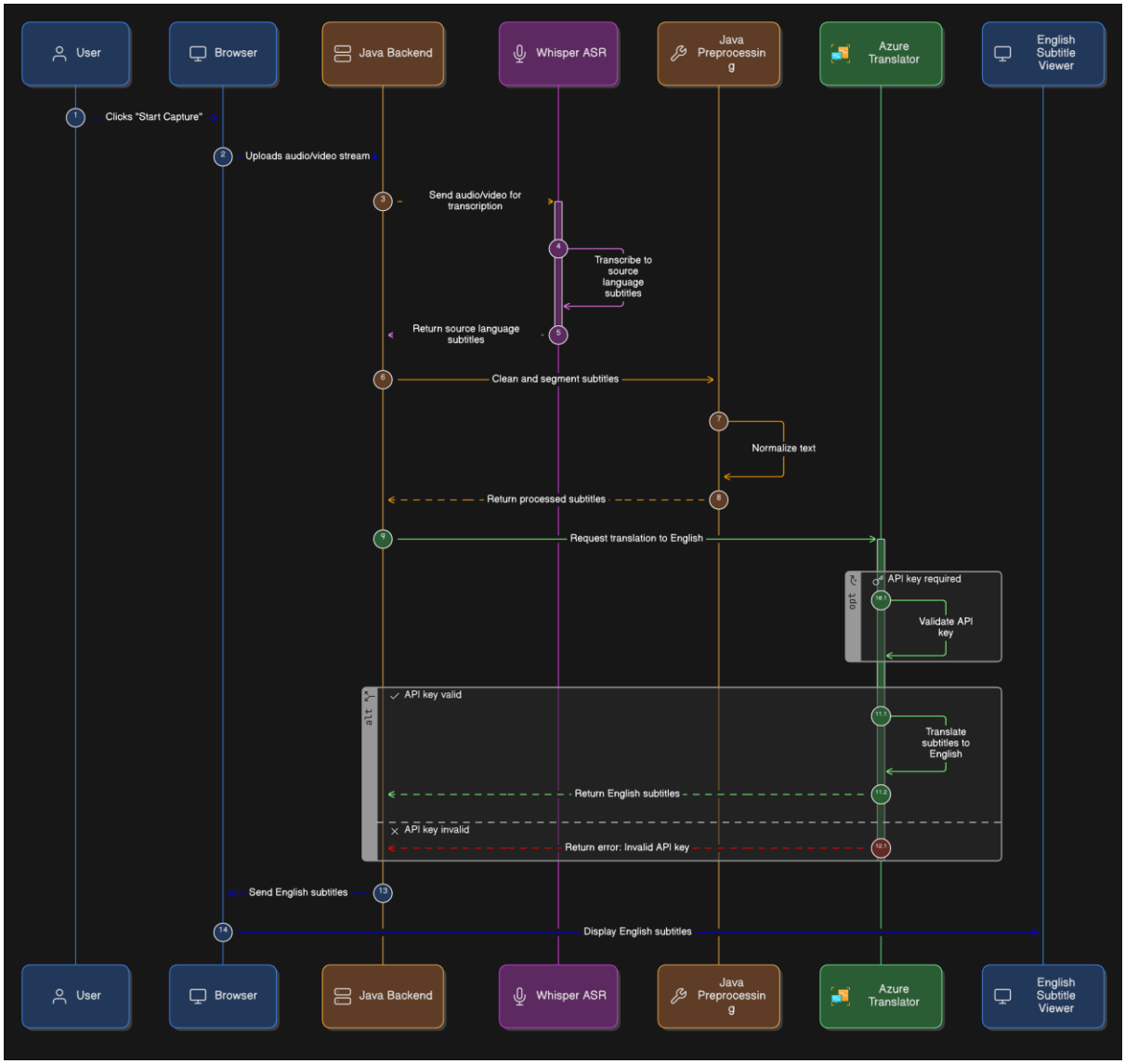
Creating a Resource Group:

1. Click on “Create a resource” on Azure home page
2. Select “Resource Group”
3. Name your resource group as “ccds-project”
4. Click on “Review + Create” and “Create”

Note: This resource group is required to be used consistently for any other resources created for this project.

2. Architecture Overview

The system is built on a modular design where each service is isolated within a shared Resource Group to ensure lifecycle management and cost-efficiency.



3. Azure Service Implementation & Configuration

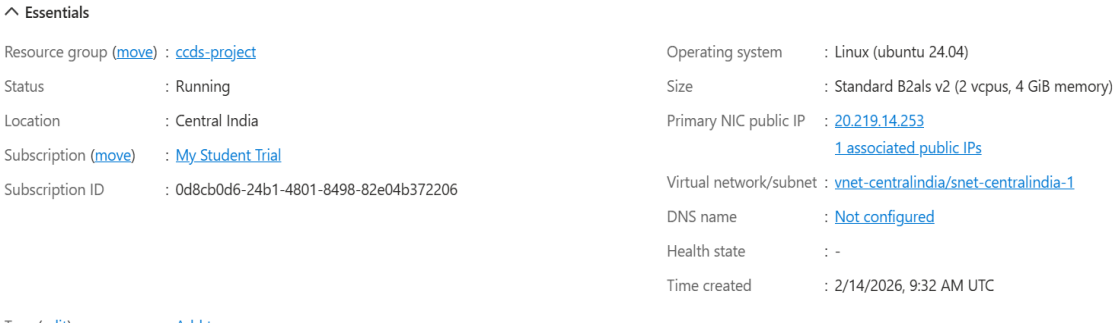
The Virtual Machine (VM) serves as the primary compute node hosting the core application logic.

- **Azure Virtual Machine Setup and Deployment**
    1. Click on “Create a resource” on Azure home page
    2. Search for “Virtual Machine” and select it
    3. Set the basic initial parameters as stated in the “Azure Basic Setup” section
    4. “Zone Options” → “Self Selected Zone”
    5. “Security Type” → “Standard”
    6. “Image” → “Ubuntu Server 24.04 LTS- x64 Gen2”
    7. “VM Architecture” → “x64”
    8. “Size” → “Standard_B2als_v2 – 2vcpus, 4GiB memory ($17.96)”
    9. “Authentication Type” → “SSH Public Key”
    10. “Username” → “azureuser”
    11. “SSH public key source” → “Generate new key pair”
    12. “SSH Key Type” → “RSA SSH Format”
    13. “Key pair name” → “whisper.vm_key”
    14. “Public inbound ports” → “Allow selected ports” → 22, 80, 443
    15. Click “Next: Disks”
    16. “OS disk size” → “Image Default (30GiB)”, “OS disk type” → “Premium SSD LRS”
    17. Select the check box “Delete with VM”, “Key management” → “Platform-managed key”
    18. Click “Next: Networking”
    19. Create a name for “virtual network” → “vnet-centralindia”, “subnet” → “snet-centralindia-1” and click on “Add”
    20. “NIC network security group” → “Basic”, “Select inbound ports” → 22, 80, 443
    21. Select checkbox “Enable accelerated networking”
    22. “Load Balancing” → “None”

Note: You may enable auto power off for your VM and set its frequency/condition/time as required.

    23. Click “Review + Create” and wait for Azure to validate your configuration settings
    24. Once validated, confirm the setup and click “Create”
    25. Wait for the VM to be successfully deployed

Note: Configured specific inbound/outbound rules via Network Security Groups (NSG) to allow traffic only on necessary ports (e.g., port 22 for SSH, port 8000 for application traffic).



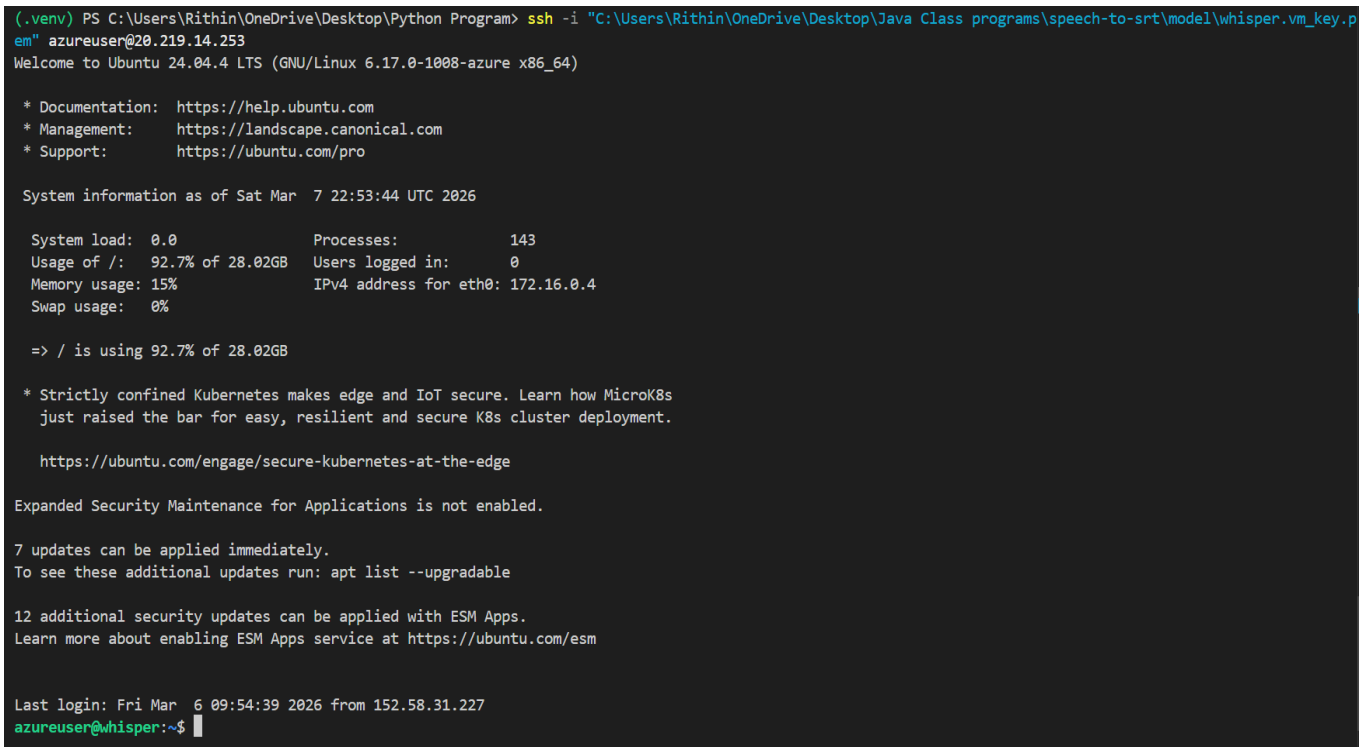
- **Connecting to VM:**
    1. After creation of “whisper-vm”, ensure it is running and “Ready” for use

2. Click on “Connect” dropdown and then click “Connect” to see a dialogue box open

Note: Since, we have enabled SSH service for our VM, we will connect to it via “Native SSH”

3. Copy the SSH command template provided as “ssh -i <private-key-file-path> azureuser@<your-vm-public-ip>”
4. Open your VS Code/CMD prompt in your local machine and paste the copied command template in the terminal/shell
5. Using the arrow keys, replace the “<private-key-file-path>” with the actual path of your private key which was provided while setting up your VM.
6. Press “Enter” and wait till your local machine connects to your VM using SSH

Note: This process of connecting to VM will have to repeated everytime the VM is stopped manually or due to timeouts.

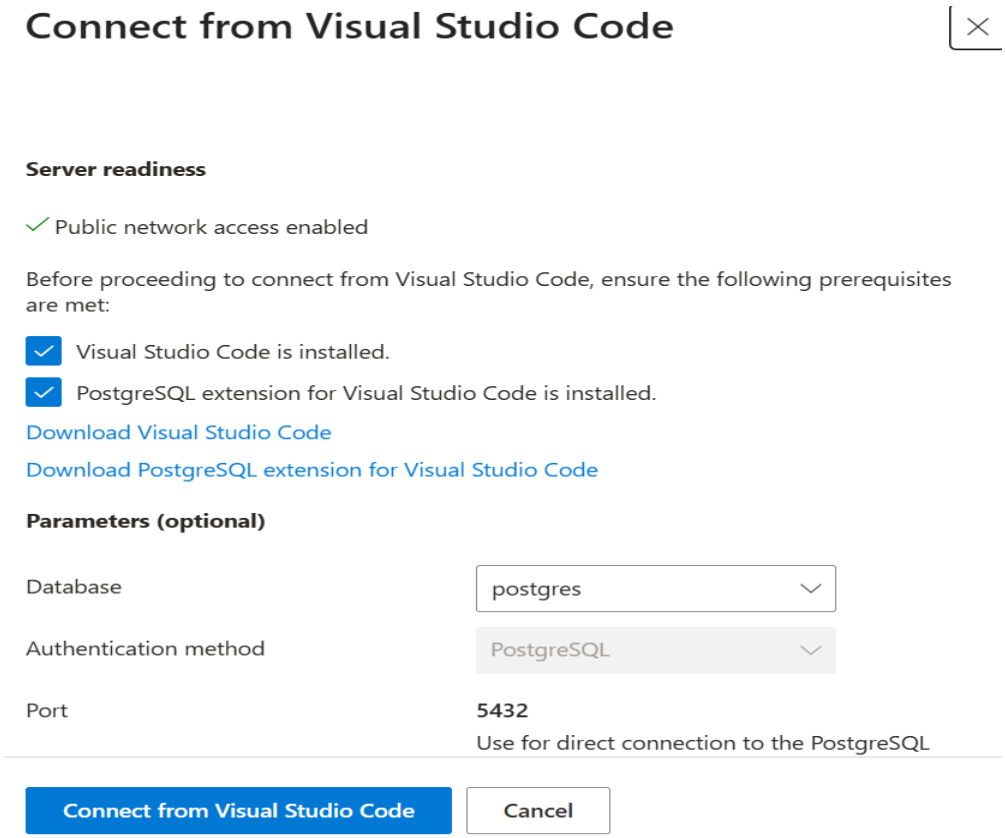


3.2 Azure Database for PostgreSQL (Flexible Server)

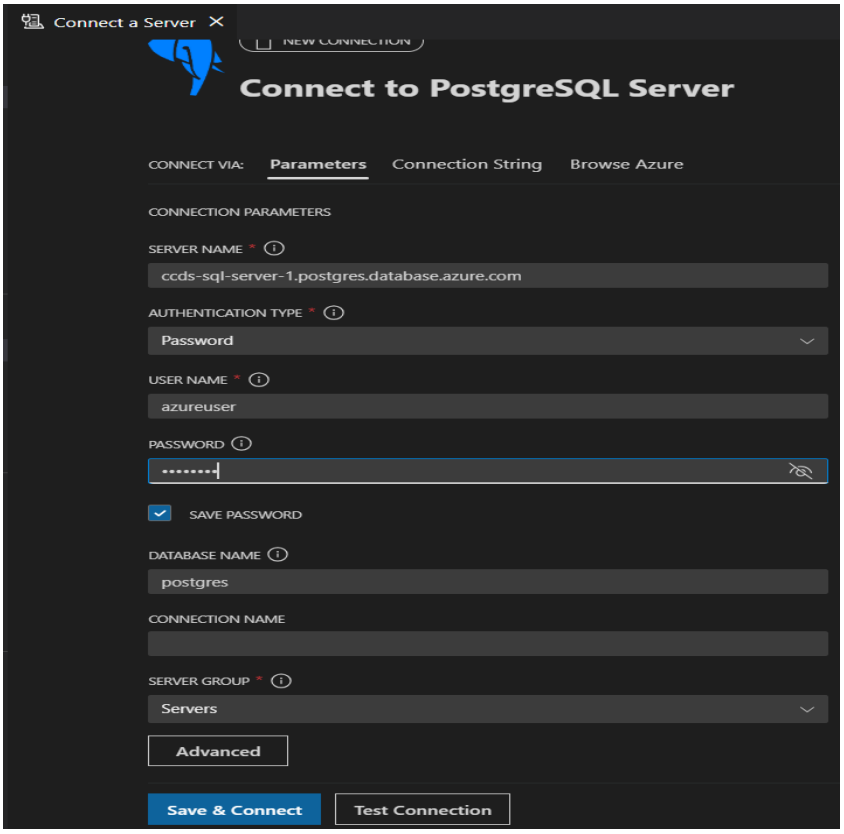
PostgreSQL is utilized to manage relational data, configured as a Flexible Server for better control over maintenance windows and high availability.

- **Connecting to VS Code:**

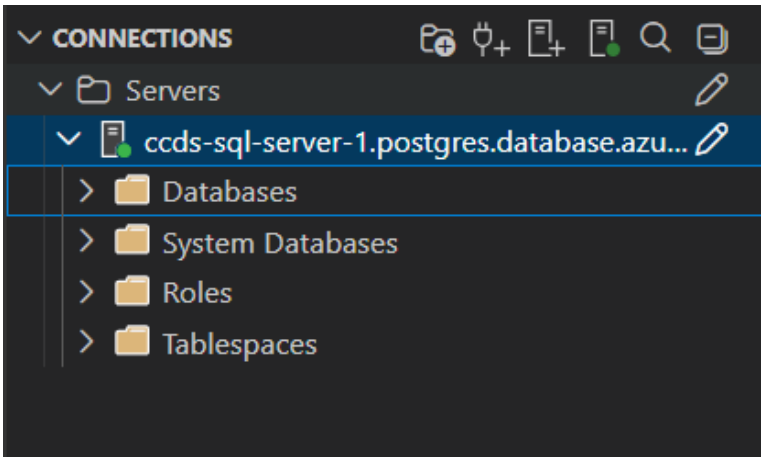
1. Navigate to your “ccds-sql-server-1”
2. Start your SQL server if it is not already running
3. After its status changes to “Ready”, navigate to “Overview” tab
4. Locate “Connect from VS Code” button on the top left corner and click it
5. Click “Connect” and select all checkboxes, choose database to work with



6. Click “Connect from Visual Studio Code” → “Open Visual Studio Code”
7. Go to your VS Code, open Azure PostgreSQL extension
8. It will automatically populate the data required to create a connection from Azure to your VS Code.

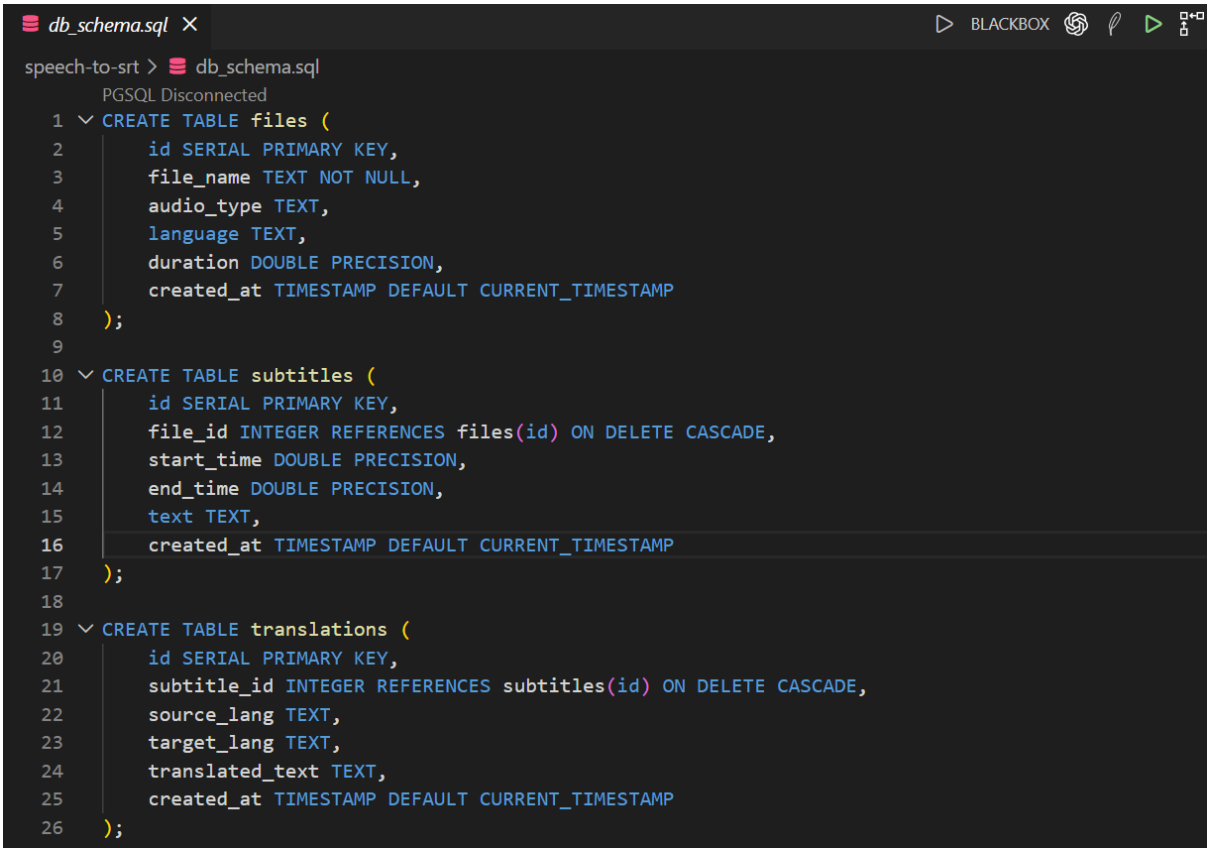


9. You need to enter your “Password” for database and click on “Test Connection”
10. Once you see a green tick mark beside “Test Connection”
11. Click “Save + Connect” and connection will be established successful.



- **DB Schema:**

1. Go to your VS Code → PostgreSQL extension
2. Navigate to your database and right click on your db_name (“postgres”)
3. Click on “New Query” → a new file will be opened
4. Write your SQL queries for creation of tables and defining the db schema



5. Save the file as “db_schema.sql”
6. Run/Execute the file all at once, the queries will be executed successfully.

- **Deployment Steps:**

1. Click on “Create a resource” on Azure home page
2. Search for “PostgreSQL for Flexible Servers” and select it
3. Set the basic initial parameters as stated in the “Azure Basic Setup” section
4. Name your server as “ccds-sql-server-1”
5. “Production Type” → “Dev/Test”
6. “Compute + Storage” → “Configure Server”
7. “Cluster” → “Server”, “Compute Tier” → “Burstable (1-20vCores)”
8. “Compute size” → “Standard_B2s (2 vCores, 4 GiB memory, 1280 max iops)”
9. “Storage Type” → “Premium SSD”, “Storage size” → “32 GiB”, “Performance Tier” → “P4 (120 iops)”
10. “Zonal Resiliency” → “Disabled (99.9% SLA)”
11. “Authentication Method” → “PostgreSQL authentication only”
12. “Administrator login” → “azureuser”, “Password” → “<your password>”, “Confirm password”
13. Click “Next: Networking’
14. “Connectivity method” → “Public access (allowed IP addresses) and Private endpoint”
15. Select the checkbox “Allow public access to this resource through the internet using a public IP address”

16. Select the checkbox “Allow public access from any Azure service within Azure to this server”
17. Click on “Add current client IP” and “Next: Security”
18. “Data Encryption key” → “Service-managed key”
19. Click “Review + Create” and “Create” post validation
20. Wait for server to be deployed

3.3 Azure Blob Storage

- **Deployment Steps:**

1. Search for “Storage Account” on the Azure home page and select it.
2. Click on “Create”
3. Set the basic initial parameters as stated in the “Azure Basic Setup” section
4. Name the storage account as “subtitleprojectstorage”
5. “Preferred storage type” → “Azure Blob Storage or Azure Data Lake Storage Gen 2”
6. “Performance” → “Standard”
7. “Redundancy” → “Locally-redundant storage (LRS)”
8. Click on “Next”
9. Select the checkboxes “Require secure transfer for REST API operations” and “Enable storage account key access”
10. “Minimum TLS Version” → “Version 1.2”
11. “Blob Storage” → “Hot”
12. Click on “Next”
13. “Public network access” → “Enable”
14. “Public network access scope” → “Enable from all networks”
15. Click on “Next”
16. Select checkboxes “Enable soft delete for blobs” → “Days to retain deleted blobs” → 7 and “Enable soft delete for file shares” → “Days to retain deleted file shares” → 7
17. Click on “Next”
18. “Encryption type” → “Microsoft-managed keys (MMK)”
19. “Enable support for customer-managed keys” → “Blob and files only”
20. Click on “Review + Create” and after validation click on “Create”

3.4 Azure Container

- **Deployment Steps:**
    1. After creation of Blob Storage called “subtitleprojectstorage”
    2. Go to “Container” section under the “Data Storage” tab on the left menu of the blob storage
    3. Click “Add Container”
    4. Name the new container as “uploads”
    5. “Anonymous Access Level” → “Private (no anonymous access)”
    6. Click “Create”

3.5 Azure AI Translator

This service is integrated to provide automated language translation capabilities.

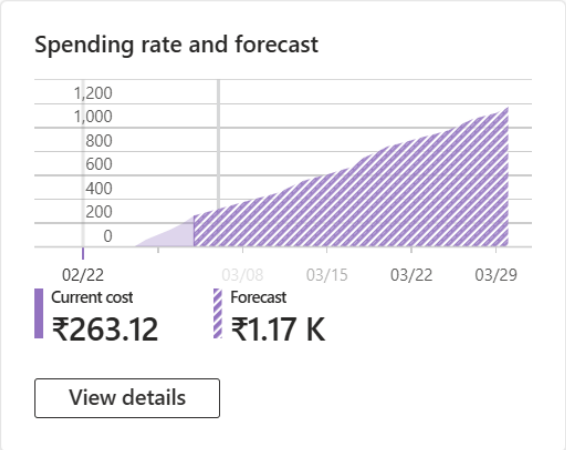
- **Deployment Steps:**
    1. Click on “Create a resource” on Azure home page
    2. Search for “translator” and select “Translator by Microsoft”
    3. Set the basic initial parameters as stated in the “Azure Basic Setup” section
    4. Name the translator as “subtitle-translator-ccds”
    5. “Pricing tier” → “Standard S1 (Pay as you go)”
    6. Click “Next”
    7. “Type” → “All networks, including the internet, can access this resource”
    8. Click “Review + Create”
    9. After validation, click “Create” to run the translator with the set configuration

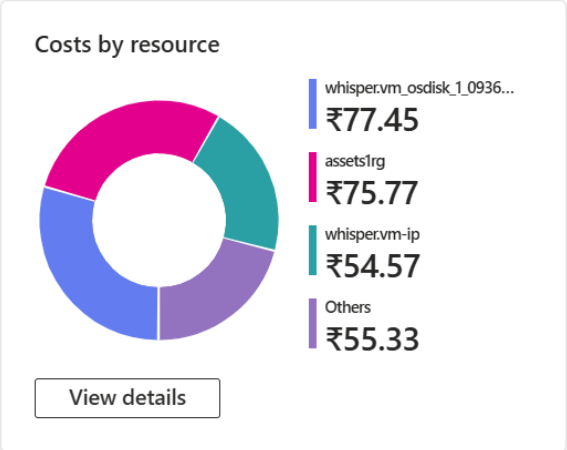
5. Project Flow/Working

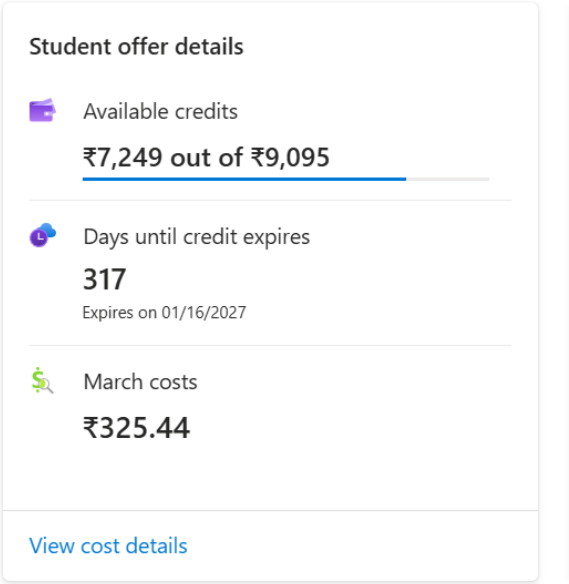
6. Summary of Configuration

The cloud architecture is configured to balance performance, cost, and security. By utilizing Managed Identities where possible, the communication between services (e.g., the VM and the Storage account) is secured without hardcoded credentials. Regular monitoring through Azure Monitor is enabled to track performance and resource health.

Price Usage Analysis:






Student offer details
Available credits
₹7,249 out of ₹9,095
Days until credit expires
317
Expires on 01/16/2027
March costs
₹325.44
View cost details